

1 Java Basics

Programming: Process of creating software

Programming languages: Tools to write software

1. **Machine language:** Primitive
Hard to understand by humans; Machine dependent
2. **High-level language** (C, C++, C#, Java, JavaScript, Pascal, Python, Visual Basic):
Easy to understand by humans; Machine independent

Java: Object-Oriented, Reliable, Secure, Portable, Multithreaded

Statement: One instruction given to the computer

Source program (Source code): Collection of statements

Source file: The file containing the source code

Interpreter / Compiler: Translate high-level language into (virtual) machine language

High-level languages are not native languages of computers.

Source program written in high-level languages cannot be understood by computer.

- **Compiler:** Translate the whole source code before execution
- **Interpreter:** Translate the source code line-by-line

Java:

1. Write the source code: Generate Java source code file (**.java**)
2. Compiler source code using the compiler "**javac**": Generate Java bytecode file (**.class**)
3. Execute the bytecode using **Java Virtual Machine (JVM)**: Generate algorithm execution results

Java Development Toolkit (JDK): A toolbox providing the environment to develop and execute Java programs, includes:

- **Java Development Tools:** Package to develop a Java program
- **Java Runtime Environment (JRE):** Package to run a Java program

Integrated development environments (IDE): Integrates editing, compiling, building, debugging, and online help

JDK can write Java program using text editor.

IDE simplifies programming by providing functionalities such as error detection. E.g. NetBeans, Eclipse, DrJava, IDEA, etc.

Package: We can divide project into multiple segments (package)

Console: Text entry and display device of a computer

Block: A pair of braces {} with the inside statements (Nested, not intersected)

If a block has only one statement, the braces can be ignored.

Class: A block with the header class.

```
public class HelloWorld {  
    ...implementations  
}
```

Each Java source file must have a same-name class block.

By convention, first letter of each word in the class name is capitalized.

(a) HelloWorld.java

Component / Member: A building block that define the class structure and behaviour

Except import and package, statements have to be placed inside a class but not necessarily in the main method.

Main method: The method being executed when executing a Java class. One of the components of the class.

One-line comment: Text starting from // until the end of the line. Ignored when program is running.

Multi-line comment: Text between /* and */

`System.out.println()` method prints something on the console and start a new line after the output.

Every statement in Java **ends with a semicolon (;)**

```
public class HelloWorld {  
    ...components  
  
    // This is the main method.  
    void main() {  
        /* This method  
           prints the sentence  
           on the console */  
        System.out.println("Hello World");  
    }  
}
```

Traditionally, the main method is a block with a header:

```
public static void main(String[] args)
```

Now, we can simplify the header to:

```
void main(String[] args)
```

The method is called `main` not `Main`.

(a) HelloWorld.java

Exercise: Personal Finance Management (PFM) Application

```
package financeMgmt;  
  
public class HomePage {  
    void main() {  
        System.out.println("Welcome, please select the service");  
        System.out.println("1. display transactions");  
        System.out.println("2. display accounts");  
        System.out.println("3. display categories");  
    }  
}
```

2 Variable

Variable: Container to store a value. Creating a variable is called **declare a variable** or **variable declaration**.

Syntax: *variableType* *variableName* = *variableValue*;

- *variableType*: Type of the value to be stored in the variable
- *variableName*: Name of the variable
- =: Assignment operator
- *variableValue*: Value to be stored

Literal: The text whose value can be inferred literally. E.g. 0, "Welcome"

Initialization: First value assignment after declaration

```
String username = "HTAKM"; // Assign a literal to a variable (initialization)
String name = username;    // Assign a variable's value to another variable (initialization)
String address;
address = "Earth";         // Assign a literal to an uninitialized variable (initialization)
name = "HU";               // Assign another literal to a variable (not initialization)
```

Identifier: Name of an element in the Java program. E.g. Class name, variable name, method name, etc.

Identifiers can contain letters (A-Z, a-z), digits (0-9), underscore (_), and \$. First character cannot be a digit.

No reserved keywords (E.g. `public`, `class`, etc.)

Identifiers are case-sensitive. No limit on the length of an identifier.

Naming conventions:

- Class name: PascalCase style (Capitalize the first letter of each word)
- Methods/Variable/Package name: camelCase style (Capitalize the first letter of each word except the first word)
- Constant name: Capitalize all letters and use underscore to separate words. E.g. `NUM_THREADS`

Primitive types:

1. Integral data: By default, integer literal is `int`. If the literal is too large, then `long`. 0 is stored by default.

Data Type	Range
<code>byte</code>	Integers from -128 (-2^7) to 127 ($2^7 - 1$).
<code>short</code>	Integers from -32768 (-2^{15}) to 32767 ($2^{15} - 1$).
<code>int</code>	Integers from -2^{31} to $2^{31} - 1$.
<code>long</code>	Integers from -2^{63} to $2^{63} - 1$.

We can specify the type by: (`long`) 0 as Java to create a value 0 using `long` format.

2. Decimal data: Integer followed by a decimal part (7.0 is decimal and 7 is integer) Also called **floating-point values**

Scientific notation: Mathematical format is in form: $a \times 10^b$. E.g. $0.0123 = 1.23 \times 10^{-2} = 0.123 \times 10^{-1}$

Java uses E or e for exponent. E.g. 0.0123 becomes 1.23e-2 or 1.23E-2

By default, decimal type is `double`.

Data Type	Range	Precision
<code>float</code>	Values from -3.4×10^{38} to 3.4×10^{38}	Limited to 6 to 7 significant decimal digits.
<code>double</code>	Values from -1.79×10^{308} to 1.79×10^{308}	Limited to 15 to 16 significant decimal digits.

3. Character data: Single character enclosed by single quotation marks. E.g. `'a'`, `'A'`, `''`

Escape sequence: Character after the backslash.

E.g. `'\t'` (Tab space), `'\n'` (End of line), `'\''` (Single quote), `'\\'` (Backslash)

The backslash is the **escape character**.

4. Boolean data: Only two Boolean values: `true` and `false`

Storage space of a computer is measured in the number of **bit**.

- 1 **byte** (B) = 8 bits
- 1 **kilobyte** (KB) = 1024 B
- 1 **megabyte** (MB) = 1024 KB
- 1 **gigabyte** (GB) = 1024 MB
- 1 **terabyte** (TB) = 1024 GB

Different data types use different space amounts:

- **byte**: 1 B, **short**: 2 B, **int**: 4 B, **long**: 8 B
- **float**: 4 B, **double**: 8 B
- **char**: 2 B
- **boolean**: 1 B typically, changes with JVM

Value can be assigned to a variable when they have the same type.

Implicit type casting: Value is automatically converted to the variable's type, with following conditions:

1. For integers and decimals, the value type's range is no larger than the variable type's range.
2. Character can be assigned to numerical variables based on ASCII code.
3. Integer literals and integer variables below 65536 can be assigned to character variables.

Explicit type casting: Explicitly specify the value's type so that the assignment can be executed.

```
/* 1.9 is double, whose range is larger than int. It cannot be assigned directly.
 * We explicitly cast 1.9 into int.
 * The value of w is 1 after type casting. Java directly drops the decimal part.
 */
int w = (int) 1.9;
```

Operator: Denotes an operation on values, variables, and objects. E.g. assignment operator =

Operands: Values, variables and objects processed by an operator. E.g. x and 1 in x = 1

Arithmetic operators:

Operator	Meaning	Example	Output
+	Addition	33 + 5	38
-	Subtraction	15.4 - 0.3	15.1
*	Multiplication	250 * 10	2500
/	Division	1.0 / 2.0	0.5
%	Remainder	23 % 3	2

Arithmetic result depends on the operand types. E.g.

Operand types	Result type
Integer, Integer	Integer
Floating-point, Floating-point	Floating-point
Integer, Floating-point	Floating-point

Augmented Assignment Operators:

Operator	Name	Example	Equivalent
+=	Addition assignment	a += 3	a = a + 3
-=	Subtraction assignment	a -= 3	a = a - 3
*=	Multiplication assignment	a *= 3	a = a * 3
/=	Division assignment	a /= 3	a = a / 3
%=	Remainder assignment	a %= 3	a = a % 3

Augmented assignment operators use implicit casting.

Increment and Decrement Operators:

Operator	Name	Description	Example	a (Initial a=1)	b
<code>++var</code>	Preincrement	Increment <code>var</code> by 1, and use new <code>var</code>	<code>int b = ++a</code>	2	2
<code>var++</code>	Postincrement	Increment <code>var</code> by 1, and use original <code>var</code>	<code>int b = a++</code>	2	1
<code>--var</code>	Predecrement	Decrement <code>var</code> by 1, and use new <code>var</code>	<code>int b = --a</code>	0	0
<code>var--</code>	Postdecrement	Decrement <code>var</code> by 1, and use original <code>var</code>	<code>int b = a--</code>	0	1

<code>int i = 10;</code>		<code>int i = 10;</code>
<code>int j = 10 * (i++);</code>	$\Longleftrightarrow$	<code>int j = 10 * i;</code>
		<code>i += 1;</code>
<code>int i = 10;</code>		<code>int i = 10;</code>
<code>int j = 10 * (++i);</code>	$\Longleftrightarrow$	<code>i += 1;</code>
		<code>int j = 10 * i;</code>

Class: Block of statements defining a group of similar real-world entities, which includes:

- **Properties, attributes, or data fields:** describe how the entities look like
- **Action, Method:** describe what the entities can do

Data fields are defined as variables outside any method or constructor.

```
public class Dog {
    String name;
    int age;
    void bark() {
        System.out.println("The dog " + name + " barks.")
    }
} // No main method, so you cannot run it.
```

Java usually divides the computer memory into three parts: Metaspace, Stack, Heap

Class instantiation: Java creates a memory segment in Heap and create variables inside it following the data field definitions.

Object: The memory segment with created class elements.

An object is an **instance** of the class. You can create multiple objects of the same class.

Reference variable: A variable to store the object address. They are stored in Stack.

Syntax: `refVariableType refVariableName = new className();`

- *refVariableType*: Should be same as *className*
- **new**: A keyword of class initialization, which tells Java to create an object.
- **()**: Cannot be omitted

Assign values to data fields:

Syntax: `refVariableName.dataFieldName = valueToBeAssigned;`

- **.: Dot operator**, which connects the container and its element.

```
Dog myDog = new Dog();
myDog.name = "John";
System.out.println("My dog is called " + myDog.name);
```

Scanner class: Provides methods to read the values input by users on the console. Defined in the package called `java.util`. Before using Scanner class, we need to import the package at the top of the file first.

```
import java.util.*; // Imports all classes in the package
import java.util.Scanner; // Only imports the Scanner class
```

To use the methods in the Scanner class, we need to create an object of it.
The Scanner class defines a `nextInt()` method to read an `int` input. It can be used repetitively.

```
Scanner input = new Scanner(System.in);
System.out.print("Please input an integer: ");
int userInput1 = input.nextInt();
System.out.println("The user input is " + userInput);
System.out.print("Please input another integer: ");
int userInput2 = input.nextInt();
System.out.println("The user input is " + userInput);
```

Methods of the Scanner object:

Method	Description
<code>nextByte()</code>	Reads an integer of the <code>byte</code> type
<code>nextShort()</code>	Reads an integer of the <code>short</code> type
<code>nextInt()</code>	Reads an integer of the <code>int</code> type
<code>nextLong()</code>	Reads an integer of the <code>long</code> type
<code>nextFloat()</code>	Reads an integer of the <code>float</code> type
<code>nextDouble()</code>	Reads an integer of the <code>double</code> type
<code>next()</code>	Reads a String (till a whitespace)
<code>nextLine()</code>	Reads a String (till the Enter key or next line)

Do not use `nextLine()` after other console-reading methods to avoid errors.

There are no methods to read a char value.

Null type: This variable does not reference to any object.

Garbage collection: If an object is not referenced by any variable, it will be cleared.

```
myDog = null;
```

3 Method Basics

Recall:

Class: Describes a group of similar real-world entities.

- Properties, attributes, or data fields: describe how the entities look like
- Action, or method: describe what the entities can do

Method: A block describing a task that can be done by this class' objects.

Method header: Statement before the block

Method body: The block (two braces and the statements inside them)

<pre>void sayHello() { System.out.println("Hello"); }</pre>	Method header: <code>void sayHello()</code> Method body: <pre>{ System.out.println("Hello"); }</pre>
---	--

Method header syntax: *returnValueType methodName parameterList*

- *methodName*: By convention, should use camelCase style
- *parameterList*: Defines all the parameters surrounded with ()

Parameter: Input required to invoke the method.

We declare parameters (parameter type and name) in the parentheses.

If multiple parameters are needed, use a comma to separate them.

Parameter list: Combination of all parameters in the parentheses.

<pre>void printSum(double number1, double number2) { double sum = number1 + number2; System.out.println("The sum is " + sum); }</pre>	Parameters: <code>double number1</code> <code>double number2</code>
---	--

Arguments (Actual parameters): Specify the values for parameters when invoking a method.

Arguments must match parameters. (Same number, compatible type)

The literals or the values stored in the arguments are passed to parameters.

Argument variables and parameter variables are not interdependent.

Changing the parameter value does not change the argument value.

```
void printSum(double number1, double number2) {  
    ...implementations  
}  
double currentNumber = 2.5;  
printSum(currentNumber, 3);
```

Method signature: Combination of method name and parameter list.

In a class, no methods should have the same signature, but they may have the same name.

```
void printSum(double number1, double number2) {  
    ...implementations  
}  
void printSum(double number, String numberStr) {  
    ...implementations  
}
```

Return value: Value returned after method invocation.

If a method returns a value, we must specify the return value type before the method name.

Return value type must be compatible with the return value. If no value is returned, the return value type is `void`.

Void Method: Return value type of the method is `void`

You can still write `return` in void methods (which returns but returns nothing)

Value-return Methods: Return value type of the method is not `void`

```
double getSum(double number1, double number2) {  
    return number1 + number2;  
}  
  
void main() {  
    Scanner scanner = new Scanner(System.in);  
    double userInput1 = input.nextDouble();  
    double userInput2 = input.nextDouble();  
    System.out.println("Their sum is " + getSum(userInput1, userInput2));  
}
```


4 Selection

Selection structure: A program executing different statements depending on certain conditions.

Syntax (One-way selection):

```
if (condition) {  
    if-statements  
}
```

- When *condition* is true, *if-statements* are executed.
- When *condition* is false, do nothing.

Syntax (Two-way selection):

```
if (condition) {  
    if-statements  
} else {  
    else-statements  
}
```

- When *condition* is true, *if-statements* are executed.
- When *condition* is false, *else-statements* are executed.

You can stack multiple if blocks.

```
boolean isGuessCorrect(int guess, int correctGuess) {  
    if (guess > correctGuess) {  
        System.out.println("Lower");  
        return false;  
    } else if (guess < correctGuess) {  
        System.out.println("Higher");  
        return false;  
    } else  
        return true;  
}
```

Boolean expression: One expression whose result is true or false.

```
boolean isCorrect = guess == correctGuess;  
if (isCorrect)  
    System.out.println("Correct");  
else  
    System.out.println("Wrong");
```

Relational operators: Commonly used to write boolean expressions.

Operator	Mathematical Symbol	Name	Example (radius is 5)	Result
<	<	less than	radius < 0	false
<=	≤	less than or equal to	radius <= 0	false
>	>	greater than	radius > 0	true
>=	≥	greater than or equal to	radius >= 0	true
==	=	equal to	radius == 0	false
!=	≠	not equal to	radius != 0	true

Due to precision issues, the arithmetic result of double value calculation may not be accurate. Try to avoid using equal to or not equal to operators.

Compound condition: Combination of multiple boolean expressions.

Operator	Name
!	not
&&	and
	or
^	exclusive or

A	!A
true	false
false	true

A	B	A && B	A B	A ^ B
true	true	true	true	false
true	false	false	true	true
false	true	false	true	true
false	false	false	false	false

Switch structure: Simplified selection structure when all conditions depend on the value of one expression.

Syntax:

```
switch (expression) {  
    case value1:  
        statements1  
        break;  
    case value2:  
        statements2  
        break;  
    :  
    default:  
        statements0  
}
```

- When *expression* is equal to *value1*, execute *statements1*.
- When *expression* is not equal to all values, execute *statements0*.

In a switch selection, Java executes the correct case until break. The default case can be omitted.

The value after the keyword **case** must be unchangeable (immutable).

If a variable is declared with a modifier **final**, this variable is called a **constant**.

Constant cannot be changed after initialization. By convention, a constant's name uses cap letters and underscores.

```
final char GRADE_A = 'A';  
switch (letterGrade) {  
    case GRADE_A: System.out.println("Excellent!");  
    case 'B': System.out.println("Keep it up!"); break;  
}
```

If letterGrade is 'A', this prints:

Excellent!

Keep it up!

Conditional Operator: Used when a variable is assigned different values depending on a condition.

Syntax: *variable* = (*condition*) ? *expressionTrue* : *expressionFalse*

- *expressionTrue* is assigned to *variableName* if *condition* is true.
- *expressionFalse* is assigned to *variableName* if *condition* is false.

It is equivalent to:

```
if (condition)  
    variable = expressionTrue;  
else  
    variable = expressionFalse;
```

```
String classStatus = (typhoneSignal < 8) ? "continue" : "cancelled";
```

5 Loops

Loop: Repeating similar or same tasks multiple times.

While loop syntax:

```
while (loop-continuation-condition) {  
    loop-body  
}
```

- As long as *loop-continuation-condition* is true, Repeat *loop-body*.
- The *loop-continuation-condition* is evaluated at the start of each loop. The loop body may be executed 0 times.

Do-while loop syntax:

```
do {  
    loop-body  
} while (loop-continuation-condition);
```

- Repeat *loop-body* until *loop-continuation-condition* is false.
- The *loop-continuation-condition* is evaluated at the end of each loop. The loop body is executed at least 1 time.

For loop syntax:

```
for (initial-action; loop-continuation-condition; action-after-each-iteration) {  
    loop-body  
}
```

This is equivalent to:

```
initial-action;  
while (loop-continuation-condition) {  
    loop-body  
    action-after-each-iteration;  
}
```

Local variable: Variable declared in a child block of the class block.

A local variable can only be used in the declaring block. A variable declared outside of any child block is a data field.

```
for (int i = 0; i < 100; i++) {  
    System.out.println("Lemons");  
}  
System.out.println(i) // This will raise error.
```

Recommended:

For loops when number of iterations are known.

Do-while loops when number of iterations are not known in advance and at least one iteration is required.

Continuation depends on user input: Terminate loop when user's input does not satisfy the continuation condition.

Sentinel value: Concrete value in a loop continuation condition to compare user input with

Sentinel-value-controlled loop: Loop that uses sentinel value in loop continuation condition

break: When Java encounters **break**, it jumps to the first statement after the loop.

continue:

When Java encounters **continue** in a for loop, it jumps to *action-after-each-iteration*.

When Java encounters **continue** in a while-loop or do-while loop, it jumps to *loop-continuation-condition*.

```
for (int i = 0; i < 5; i++) {  
    if (i % 2 == 1)  
        continue;  
    System.out.print(i);  
}  
System.out.println("hmm");
```

Print: 024hmm

```
for (int i = 0; i < 5; i++) {  
    if (i % 2 == 1)  
        break;  
    System.out.print(i);  
}  
System.out.println("hmm");
```

Print: 0hmm

6 Arrays

Array: Collection of data with the same data type inside a pair of braces {}

Syntax: *elementType* [] *refVariableName* = {*element1*, ..., *elementN*};

- *element1*, ..., *elementN* must have the type *elementType*.
- *elementType* can be primitive or reference types.
- []: This variable stores an array instead of a single value.
- An array is an object, so the variable is a reference variable.

Syntax: *elementType* [] *refVariableName* = new *elementType* [*numOfElements*];

- *new*: Create a new array object
- In the array, all elements are 0 if *elementType* is integer type, 0.0 for decimal type, '\u0000' for char, false for bool, and null for reference type.

Once an array is created, its size is fixed. You cannot add or delete elements from an array.

Syntax for getting the length of the array: *refVariableName*.length

Syntax for accessing elements through index: *refVariableName* [*elementIndex*]

- The indices start from 0 to *arrayVariable*.length - 1
- The syntax gives the element at the (*elementIndex* + 1)-th position.

Arrays are reference-type and created in Heap. The array reference variable stores the Heap address.

Therefore, we need to access the elements first before printing.

```
int[] newArray = {1, 5, 3, 6, 3};
System.out.println(newArray); // This prints the reference (Heap address), not the elements
for (int i = 0; i < newArray.length; i++)
    System.out.print(newArray[i]);
```

We can also access the elements via a foreach loop.

Foreach loop syntax:

```
for (elementType variableName : refVariableName) {
    loop-body
}
```

- This sequentially assign elements in *refVariableName* to *variableName* and repeat the loop body.
- It cannot be used to change element values.

```
double[] newArray = {1.2, 5.1, 3.2, 6.5, 3.7};
for (double number : newArray)
    System.out.print(number);
for (int i = 0; i < newArray.length; i++)
    newArray[i] *= 2;
```

Copying arrays: Syntax for `arraycopy()`: `arraycopy(srcArray, src_pos, destArray, dest_pos, length)`

- *src_pos*: Starting position of *srcArray* to start copy from
- *dest_pos*: Starting position of *destArray* to start copy to
- *length*: Number of array elements to be copied

```
int[] list1 = {1, 5, 3, 6, 3};
// This makes list2 reference to list1.
// Change list2 will also change list1.
int[] list2 = list1;

// Use for loop to copy
int[] list3 = new int[list1.length];
for (int i = 0; i <= 2; i++)
    list3[i] = list1[i];

// We can also use arraycopy
int[] list4 = new int[list1.length];
arraycopy(list1, 0, list4, 0, 3);
```

Pass-by-reference: For reference variables like array, the value stored in the argument is the Heap location of the array. Changing the elements in the array parameter will also change the elements in the array argument.

```
void main() {
    int[] array = {1, 2, 3, 4};
    doubleTheArrayElements(array); // array is changed as well
}
void doubleTheArrayElements(int[] array) {
    for (int i = 0; i <= array.length; i++)
        array[i] *= 2;
}
```

Two-dimensional array: Nested array or an array of arrays.

Ragged array: The rows or child arrays have different lengths.

Syntax: `elementType [] [] refVariableName = {childArray1, ..., childArrayM}`

- `[] []`: Variable type is a two-dimensional array of elements with type *elementType*.
- `[] [] []` means a three-dimensional array, etc.

Syntax: `elementType [] [] refVariableName = new elementType [numOfChildArrays] [numOfElementsInEachChildArray]`

- **new**: Create a new array object. Default is similar to one-dimensional array.
- Each row or child array must have the same length. Ragged array cannot be created this way.

Syntax for creating ragged array:

```
elementType [] [] refVariableName = new elementType [numOfChildArrays] []
refVariableName [0] = new elementType [numOfElementsInChildArray0];
⋮
```

Syntax for accessing elements: `refVariableName [childArrayIndex] [elementIndexInChildArray]`

Syntax for getting length:

- `refVariableName.length`: Size of parent array, which is number of child arrays
- `refVariableName [childArrayIndex].length`: Size of child array indexed by *childArrayIndex*

7 String

String: Sequence of characters. It is an object (reference data type) in Java.

Syntax (Two methods are the same):

- `String stringRefVariableName = new String(stringLiteral);`
- `String stringRefVariableName = stringLiteral;`

String literal: Literal for strings, but not a primitive-type literal. E.g. "Hello World"

Using first method creates a new String, while using second method stores the Heap address of the String literal.

Strings are immutable. Changing any character in the String will cause Java to create a new String.

Therefore, second method is recommended to avoid creating the same String multiple times.

Method of the String object:

Method	Description
<code>length()</code>	Returns the number of characters in this string
<code>charAt(index)</code>	Returns the character at the specified index from this string
<code>concat(s1)</code>	Returns a new string that concatenates this string with string <code>s1</code>
<code>toUpperCase()</code>	Returns a new string with all letters in uppercase
<code>toLowerCase()</code>	Returns a new string with all letters in lowercase
<code>trim()</code>	Returns a new string with whitespace characters trimmed on both sides

1. `stringRefVariable.length()`: Number of characters in the string.
Double quotations are not counted. Escape sequence and whitespace are counted as one characters.
It is a method, not a data field. Therefore, parentheses are necessary.
2. `stringRefVariable.charAt(index)`:
First character is indexed by 0 and last character is indexed by `stringRefVariable.length() - 1`
This can be used to get character from user input after using `nextLine()` method.
3. `stringRefVariable1.concat(stringRefVariable2)`:
Equivalent to `stringRefVariable1 + stringRefVariable2`
4. `stringRefVariable.trim()`: Eliminate whitespace characters from both ends of the string.
Whitespace characters: ' ', \t, \f, \r, \n, etc.

Comparing Strings: Do not use relational operators to compare strings. Instead, use following methods.

Method	Description
<code>equals(s1)</code>	Returns true if two strings are equal
<code>equalsIgnoreCase(s1)</code>	Returns true if two strings are equal, case insensitive
<code>compareTo(s1)</code>	Returns difference in ASCII codes of first indifference
<code>compareToIgnoreCase(s1)</code>	Returns difference in ASCII codes of first indifference; case insensitive
<code>startsWith()</code>	Returns true if this string starts with <code>s1</code>
<code>endsWith()</code>	Returns true if this string ends with <code>s1</code>
<code>contains()</code>	Returns true if <code>s1</code> is a substring in this string

1. `s0.equals(s1)` / `s0.equalsIgnoreCase(s1)`: Checks if `s0` and `s1` have the same content.
`s0 == s1` checks if heap address is the same, not the content.
It may return false when two same content string variables have different heap address.
2. `s0.compareTo(s1)` / `s0.compareToIgnoreCase(s1)`: Identifies minimum index of indifference and returns the offset.
E.g. `s0 = "abcne"` and `s1 = "abxkm"`: `s0.compareTo(s1)` returns ASCII code of 'c' minus the ASCII code of 'x'.
`s0.compareToIgnoreCase(s1)` converts both `s0` and `s1` into lower case first before calculating ASCII difference.

Methods to find a character or substring in a String (Returns -1 if not matched):

Method	Description
<code>indexOf(ch)</code>	Returns index of the first occurrence of character in this string
<code>indexOf(ch, fromIndex)</code>	Returns index of the first occurrence of character in this string after <code>fromIndex</code>
<code>indexOf(s)</code>	Returns index of the first occurrence of string in this string
<code>indexOf(s, fromIndex)</code>	Returns index of the first occurrence of string in this string after <code>fromIndex</code>
<code>lastIndexOf(ch)</code>	Returns index of the last occurrence of character in this string
<code>lastIndexOf(ch, fromIndex)</code>	Returns index of the last occurrence of character in this string before <code>lastIndex</code>
<code>lastIndexOf(s)</code>	Returns index of the last occurrence of string in this string
<code>lastIndexOf(s, fromIndex)</code>	Returns index of the last occurrence of string in this string before <code>lastIndex</code>

Conversion between strings and numbers: Error if not numerical string.

Syntax:

```
intVariableName = Integer.parseInt(intString);
```

```
doubleVariable = Double.parseDouble(doubleString);
```

- First line converts a String to an integer.
- Second line converts a String to a double value.

Placeholder: Specify the type of values to be filled later in the string

Placeholder Specifier	Value to be replaced
<code>%b</code>	A Boolean
<code>%c</code>	A character
<code>%d</code>	A decimal integer
<code>%f</code>	A floating-point number
<code>%e</code>	A number in scientific notation
<code>%s</code>	A string

`System.out.printf()`: Format the output with placeholder

Syntax: `System.out.printf(stringWithPlaceholder, value1, ..., valuek)`

- Each placeholder is filled by corresponding value.

You can also add numbers to specify the output width of the placeholder. E.g.

- `%8d`: Right-aligned decimal integer and width is 8
- `%-9d`: Left-aligned decimal integer and width is 9
- `%7.1f`: Right-aligned floating-point number with 1 digit after the decimal point and width is 7

If the specified width is less than the length of the value, the specified width is ignored and the width is expanded to fit the value.

8 Methods: Advanced

Method Overloading: Have two methods with same name but different parameter lists. Java automatically chooses the method whose parameter list best matches the arguments. Error will be raised if there are no compatible methods.
Some notes:

- We cannot overload a method by only changing the return type.

```
void func(int i, int j) { System.out.println("First"); }
void func(int i, double j) { System.out.println("Second"); }
void func(int i, int j, int k) { System.out.println("Third"); }
void main(String[] args) {
    func(1, 5);    // First
    func(1, 5.2); // Second
    func(2, 5, 2); // Third
    func(2.5, 1); // Error
}
```

Pass-by-value: Only the value of the arguments are passed to the parameters

Primitive data: Different activation record, value is copied.

Reference data: Different activation record, heap address (reference) is copied (Pass-by-reference)

Variable arguments: A method taking an arbitrary number of arguments (indicate with ellipsis ...)

```
public static int sum(int... numbers) {
    int total = 0;
    for (int n : numbers)
        total += n;
    return total;
}
void main() {
    System.out.println(sum(2, 4, 6));    // 12
    System.out.println(sum(2, 4, 6, 8)); // 20
    System.out.println(sum());           // 0
}
```

Recursion: Method invoking itself

```
public static int factorial(int number) {
    if (number <= 0)
        return 1;
    return number * factorial(number - 1);
}
void main() {
    System.out.println(factorial(2)); // 2
    System.out.println(factorial(10)); // 3628800
}
```


9 Classes

Constructor: Special method whose name must be the same as the class name.

Syntax:

```
className (parameterListIfNeeded) {  
    constructorBody  
}
```

- Constructors do not have return statements and return value types.

If no constructors are explicitly defined in the class, java automatically creates a **default constructor**.

Syntax: `className () {}`

- Default constructor requires no parameters and has no statements.

A constructor is invoked automatically when an object is instantiated using the `new` keyword.

```
public class Dog {  
    String name;  
  
    Dog(String newName) {  
        name = newName;  
    }  
}  
  
public class Zoo {  
    void main() {  
        Dog dog = new Dog();  
    }  
}
```

Constructor Overloading: Constructors can be overloaded by having constructors with different parameter lists.

Java automatically chooses the compatible constructor, and raises an error if no appropriate constructors can be found.

To invoke another constructor, we can use keyword `this`. Only one constructor can be invoked in another constructor. Syntax:

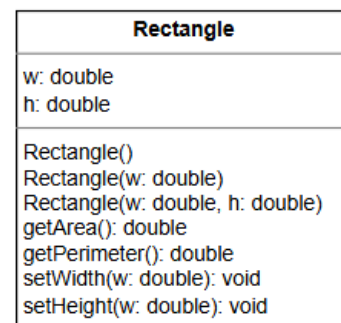
`this(argumentsIfNeeded)`

- Similarly, `this` can be used to reference the data field.
- Only one constructor can be called inside each constructor.

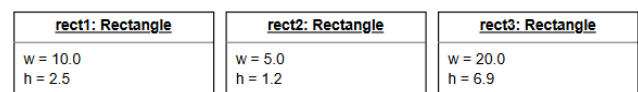
Unified Modeling Language (UML) notation: Standardized way to illustrate a class and objects.

```
public class Rectangle {  
    double w;  
    double h;  
  
    Rectangle() {  
        System.out.println("New Rectangle");  
    }  
    Rectangle(double w) {  
        this(w, w);  
    }  
    Rectangle(double w, double h) {  
        this();  
        this.w = w;  
        this.h = h;  
    }  
  
    double getArea() {return w * h;}  
    double getPerimeter() {return 2 * (w + h);}  
    void setWidth(double w) {this.w = w;}  
    void setHeight(double h) {this.h = h;}  
}
```

(a) Code



(b) UML Class Diagram



(c) UML Object Diagram

Static data fields: Shared by all objects of this class. Declared with the **static** modifier.

Access syntax: *className.dataField*, *objectName.dataField* (not recommended)

- Using **this** to access static data field is also not recommended.
- The variables now can be classified into three categories:
 1. **Class/Static data field:** Declared with **static** in a class yet outside any child blocks
It is shared by all objects and accessed throughout the entire class.
 2. **Member/Instance data field:** Declared without **static** in a class outside of any child blocks
It is unique to each object and accessed throughout the entire class.
 3. **Local variable:** Declared within a child block
Accessed from its declaration to the end of its declaring child block.
- No two data fields can have the same name, but a data field and a local variable can have the same name.
- Local variables can have the same name as long as their scopes do not overlap.

Static methods: Methods that do not require the state of a particular object. Declared with the **static** modifier.

Access syntax: *className.methodName(args)*, *objectName.methodName(args)* (not recommended)

- Main method can be non-static, but it is recommended to be static.
- **Factory method:** Static method that invokes constructors
- Initializing objects is not needed to use static method.
- Static methods can only access static data fields. Non-static methods can access both data fields.
- It is recommended that rather than non-static methods, static methods are used to handle static data.

```
public class Circle {
    static final double PI = 3.14159;
    static int count = 0;
    double radius;
    Circle(double radius) {
        this.radius = radius;
        count++;
    }

    // Factory method
    static Circle fromRadius(double radius) {
        return new Circle(radius);
    }
    static int getCount() {return count;}
    double getArea() {
        // Math.pow(...) is a static method in Math class.
        // If it is not static, we need to create a new Math object, which is undesirable.
        return PI * Math.pow(radius, 2);
    }
    static void main() {
        Circle c1 = new Circle(2);
        System.out.println(Circle.getCount()) // 1
        System.out.println(c1.getCount())     // 1 (not recommended)
        Circle c2 = new Circle(4);
        Circle c3 = Circle.fromRadius(5);
        System.out.println(Circle.count);     // 3
        System.out.println(c2.count);         // 3 (not recommended)
        System.out.println(c3.getArea());     // 78.53975
    }
}
```

In large project, classes are usually divided into different packages for easier code management. We can change the visibility of a class or its member by adding visibility modifiers.

- **Public:** Declare with the **public** modifier. (+ in UML)
The class or its member can be accessed by classes in other packages.
- **Protected:** Declare with the **protected** modifier. Related to superclass or subclass. (# in UML)
Class cannot be declared protected.
- **Default:** Declare without any modifier. (~ in UML)
The class or its member can only be accessed by classes in the same package.
- **Private:** Declare with the **private** modifier. (- in UML)
Class member can be accessed only by its class and cannot be accessed by other classes, even in the same package.
Class cannot be declared private.

Reasons why we need to make data fields private:

1. Security: Some data fields are only for internal logic and shouldn't be known by users.
2. Immutability: Users can see the data fields but shouldn't be able to change them.
Accessor (getter) method: Non-private method that returns the value of private data field (in different format)
3. Data validation: Prevent invalid change with dot operator
Mutator/Setter: Non-private method that accepts a parameter and users it to update the private field's value.

Data field encapsulation: Add corresponding accessor and mutator methods if private data fields need access or change.

Reasons why we need to make methods private:

1. Cleaness: Reduce the methods displayed to users.
2. Security: Stop users from invoking the method inappropriately.
3. Maintainability: We can change this method when necessary without worry of causing errors on code from other classes.

With visibility modifiers, we can control what users can see and how users can change.

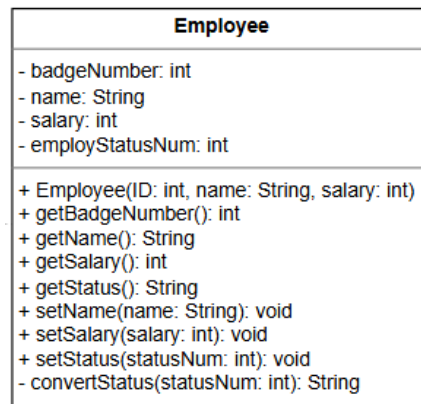


Figure 10: UML Class Diagram with modifiers

10 Input/Output

File class: Class used to handle objects for files or file folders.

Create a File object for the given path: `File fileVar = new File(filePath);`

- `fileVar.isDirectory()`: Check whether the object represents a directory (file folder).
- `fileVar.isFile()`: Check whether the object represents a file.
- It also contains methods for obtaining properties of a file/directory, renaming and deleting the file/directory.

PrintWriter class: Class that is used to write data to a file.

Before using the PrintWriter class, we need to import the package at the top of the file first.

```
import java.util.Scanner;    // Only imports the Scanner class
import java.io.PrintWriter; // Only imports the PrintWriter class
```

To use the methods in the PrintWriter class, we need to create an object of it.

After creating the PrintWriter class, we can use `print`, `println` and `printf` methods to write data.

`PrintWriter(System.out)` is just the standard console output. Finally, use the `close()` method to close the object.

```
Scanner input = new Scanner(System.in);           // Read the data
// Read the data and do something
PrintWriter output = new PrintWriter(System.out); // Write the data
// Overwrite the data in the file
```

However, when we want to create a Scanner object or a PrintWriter object with a File object, it may throw an error.

Therefore, every method that creates them need to tell Java that the **Exception class** is needed.

After finishing everything, we must close the object by calling `close()` method.

Syntax: *modifiers returnType methodName(parameterList) throws Exception {...}*

```
public static void write(String s) throws Exception {
    // Using print from PrintWriter(System.out) is the same as regular print methods
    File file = new File("log.txt");
    PrintWriter writer = new PrintWriter(file);
    writer.println(s); // Write string to log.txt file
    writer.close();
}
```

We can ensure that we close the object with the **try-with-resources** structure.

Syntax: `try (`

```
    initializeStatements
) {
    statementsAfterInitialization
}
```

```
File file = new File("log.txt");
try (Scanner sc = new Scanner(file)) {
    while (sc.hasNext()) {
        System.out.println(sc.next());
    }
}
```

```
File file = new File("log.txt");
try (PrintWriter pw = new PrintWriter(file)) {
    pw.print(...);
}
```

Token: A data point.

Delimiter: The string signalling the end of a token (default: space)

The Scanner object has a method `useDelimiter(string)` to change the delimiter to string. Reset with `reset()`.

```
File file = new File("data.txt");
Scanner sc = new Scanner(file);

sc.useDelimiter(".|\\n"); // Separators are either , or newline \n

while (sc.hasNext()) {
    System.out.println(sc.next());
}
sc.close();
```

PrintWriter and Scanner classes are text I/O, which directly input and output text.

InputStream and OutputStream are binary I/O, which convert text to ASCII before writing or ASCII to text after reading.

Usually, binary I/O is more efficient than text I/O.

FileInputStream class / FileOutputStream class: Read/write only byte data from/to file.

Again we need to first import the package at the top of the file first.

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
```

Similar to scanner and PrintWriter, we need to throw exceptions.

We can create a FileInputStream object to read tokens one-by-one from a file using the `read()` method.

If all data has been read, the next value read from the file is -1.

We can also use `available()` method to returns the number of remaining tokens.

We can then create a FileOutputStream object to write into a file using the `write()` method.

```
try (FileInputStream input = new FileInputStream(new File("original.txt"));
    // Second arg: true means append to the end, false means new content replaces current content
    FileOutputStream output = new FileOutputStream(new File("copy.txt"), true)) {

    // Append the data from original.txt to copy.txt
    int i;
    while (input.available() > 0) { // This can also be done by (i = input.read()) != -1
        i = input.read();
        output.write(i);
    }
}
```


DataInputStream class / DataOutputStream class: Handle more types of data.

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
```

We can create a DataInputStream object from FileInputStream and a DataOutputStream object from FileOutputStream. Other methods are similar to the FileInputStream/FileOutputStream.

Method	Description
<code>readBoolean()</code>	Reads a Boolean from the input stream
<code>readByte()</code>	Reads a byte from the input stream
<code>readChar()</code>	Reads a character from the input stream
<code>readFloat()</code>	Reads a float from the input stream
<code>readDouble()</code>	Reads a double from the input stream
<code>readInt()</code>	Reads an integer from the input stream
<code>readLong()</code>	Reads a long from the input stream
<code>readShort()</code>	Reads a short from the input stream
<code>readLine()</code>	Reads a line from input
<code>readUTF()</code>	Reads a String in UTF format

Table 3: Methods of DataInputStream

Method	Description
<code>writeBoolean(b)</code>	Writes a Boolean to the output stream
<code>writeByte(i)</code>	Writes the eight low-order bits of the argument to the output stream
<code>writeBytes(s)</code>	Write the lower byte of the characters in a string to the output stream
<code>writeChar(ch)</code>	Writes a character (2 bytes) to the output stream
<code>writeChars(s)</code>	Write every character in the string to the output stream, 2 bytes per character
<code>writeFloat()</code>	Writes a float to the output stream
<code>writeDouble()</code>	Writes a double to the output stream
<code>writeInt()</code>	Writes an integer to the output stream
<code>writeLong()</code>	Writes a long to the output stream
<code>writeShort()</code>	Writes a short to the output stream
<code>writeUTF()</code>	Writes a String in UTF format

Table 4: Methods of DataOutputStream

```
try (DataInputStream din = new DataInputStream(new FileInputStream("original.txt"));
    DataOutputStream dout = new DataOutputStream(new FileOutputStream("copy.txt"))) {...}
```

BufferedInputStream class / BufferedOutputStream class: Efficient in handling large byte datasets.

```
import java.io.BufferedInputStream;
import java.io.BufferedOutputStream;
```

Methods are the same as FileInputStream/FileOutputStream.
Maintains an internal buffer instead of reading one byte at a time.

```
try (BufferedInputStream bin = new BufferedInputStream(new FileInputStream("original.txt"));
    BufferedOutputStream bout = new BufferedOutputStream(new FileOutputStream("copy.txt"))) {...}
```


11 Inheritance and Polymorphism

Three pillars of object-oriented programming: Encapsulation (Ch.9), inheritance, and polymorphism.

Superclass/Parent class/Base class: Class which other classes inherit properties and behaviours from.

Subclass/Child class/Extended class/Derived class: Inherits properties and behaviours from another class.

Supertype: A variable's type defined by its superclass.

Subtype: A variable's type defined by a subclass.

Syntax: `public class subClass extends superClass`

Subclass does not inherit superclass's constructors.

We use `super()` with arguments if necessary to invoke superclass constructor from subclass constructor.

If explicitly invoked, it must be the first statement. If not, it will automatically put it with no args as the first statement.

```
public class Vehicle {
    String brand;
    Vehicle(String brand) {
        this.brand = brand;
    }
}
public class Car extends Vehicle {
    String model;
    Car(String brand, String model) {
        super(brand);
        this.model = model;
    }
}
```

A subclass inherits non-private data fields and methods from the superclass.

Protected: Declared with the `protected` modifier.

Protected members in a public class can be accessed by classes in the same package and its subclasses in different packages.

Visibility: `public > protected > default > private`

Modifiers	Accessed From			
	Same class	Same package	Subclass	Different package
<code>public</code>	✓	✓	✓	✓
<code>protected</code>	✓	✓	✓	×
<code>default</code>	✓	✓	×	×
<code>private</code>	✓	×	×	×

Method Overriding: Define a method in the subclass with the same signature of a method in a superclass.

Subclass method must have the same or compatible return type of the method in the superclass.

Method overriding cannot weaken the method visibility.

E.g. If the method is protected in the superclass, then it must be protected or public in the subclass.

We can invoke the method from superclass in subclass by `super.methodName()`.

Some notes:

- Static method cannot be overridden but it can be inherited.
Defining a static method with same signature in the subclass is not overriding.
- Private method cannot be overridden and inherited.
Defining a private method with same signature in the subclass is not overriding.

Method Overloading can be performed with same name but different parameter lists in different classes related by inheritance.

Object class: Base superclass of all Java classes (any type except primitive data types)

If no inheritance is specified when defining a class, the immediate superclass is Object.

Object has `toString()` that returns a string representation of the object, which we can override.

```
public class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override    // Inform the system that you intend to override, optional but best practice
    public String toString() {
        return "Person[name: " + name + ", age = " + age + "]";
    }
}
```

Polymorphism: A reference variable of a superclass can store an object of the subclass.

Dynamic method dispatch: Invokes the method of the object referenced by the variable.

- The variable class must still contain or inherit the method.
- If the variable type and the object type are the same class, invoke the method defined in that class.
- If the variable type is a superclass and the object type is a subclass, invoke the method defined in the subclass.
- If the object type does not have this method, check if the method can be inherited from the superclass of the object.

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound.");
    }
}
class Mammal extends Animal {}
class Cat extends Mammal {
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
}
public class Main {
    public static void main() {
        Animal animal1 = new Cat(...);    // Cat is subclass, Animal is superclass.
        Animal animal2 = new Mammal(...); // Mammal is subclass, Animal is superclass.
        Mammal animal3 = new Cat(...);    // Cat is subclass, Mammal is superclass.
        Object animal4 = new Cat(...);    // Cat is subclass, Object is superclass.

        animal1.makeSound(); // Meow
        animal2.makeSound(); // Animal makes a sound.
        animal3.makeSound(); // Meow
        animal4.makeSound(); // Error, Object class does not have the method makeSound()
    }
}
```

Static methods are not dynamically dispatched.

If the variable type is a superclass and the object type is a subclass, invoke the static method defined in the superclass.

We can test whether an object is an instance of a class (including superclasses) by using the `instanceof` operator. If it is an instance of a class, the object can be cast into that class. Casting does not change the object type. We can also use `instanceof` to create and assign a new variable of the compatible type.

```
class Animal {
    void makeSound() {
        System.out.println("Animal makes a sound.");
    }
    ...
}
class Mammal extends Animal {...}
class Cat extends Mammal {
    @Override
    void makeSound() {
        System.out.println("Meow");
    }
    ...
}
public class Main {
    static void makeSound(Object object) {
        if (object instanceof Animal animal) {    // This combines checking and casting
            animal.makeSound()    // Dynamic method dispatch still happens
        }

        public static void main() {
            Object catObject = new Cat(...);

            System.out.println(catObject instanceof Cat);    // true
            System.out.println(catObject instanceof Mammal); // true
            System.out.println(catObject instanceof Animal); // true
            System.out.println(catObject instanceof Object); // true

            Object mammalObject = new Mammal(...);

            System.out.println(mammalObject instanceof Cat);    // false
            System.out.println(mammalObject instanceof Mammal); // true

            Mammal animal3 = (Mammal) catObject;    // catObject is an instance of Mammal, allowed
            Cat animal4 = (Cat) mammalObject;        // mammalObject is not an instance of Cat, error

            makeSound(new Cat());    // Meow
            makeSound(new Mammal()); // Animal makes a sound.
        }
    }
}
```


12 Graphical User Interface (GUI)

We can build a Java program with GUI using **JavaFX**, which contains a set of graphics and media packages.

We can use JavaFX Scene Builder or other tools to create GUI of applications.

To use JavaFX, the class must extend the **Application class**, which is an **abstract class**.

Subclasses of an abstract class must override all abstract methods in the superclass (discussed in next chapter).

Application class contains one abstract method **start(Stage primaryStage)**:

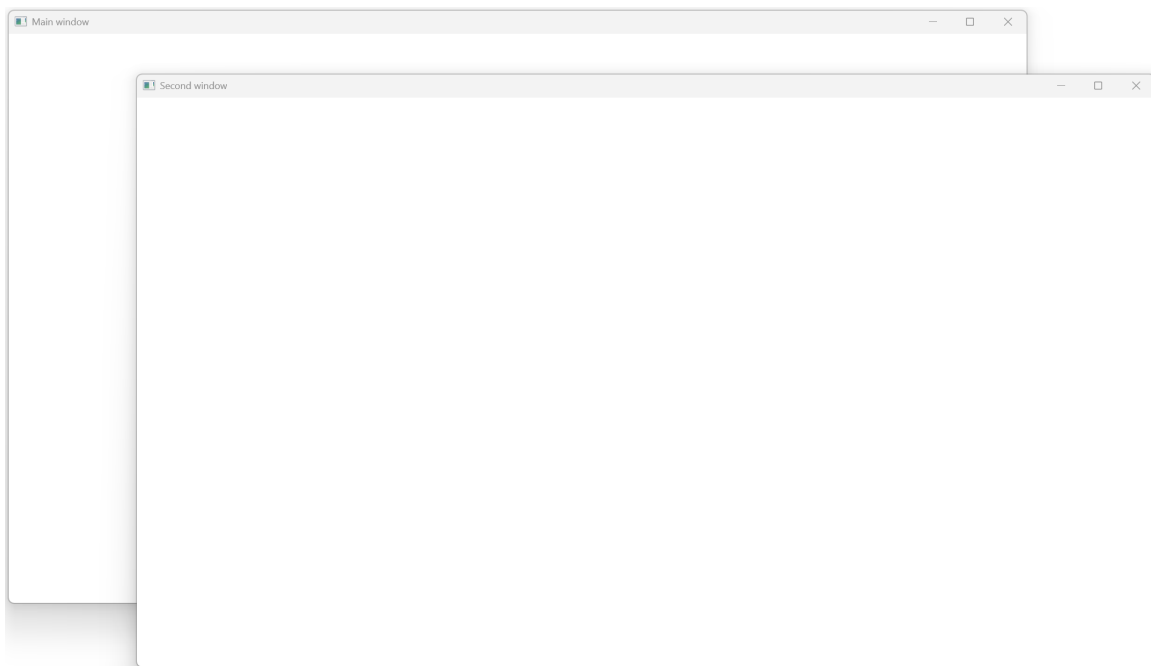
- **Stage class** is the top level JavaFX container. A stage object is a window.
- Multiple stages can be created for switching windows or adding subwindows.

```
// Import necessary packages
import javafx.application.Application;
import javafx.stage.Stage;

public class Demo extends Application {
    @Override // Override the abstract method in Application class
    public void start(Stage primaryStage) {
        primaryStage.setTitle("Main window"); // Set the title
        primaryStage.show();                  // Display the stage

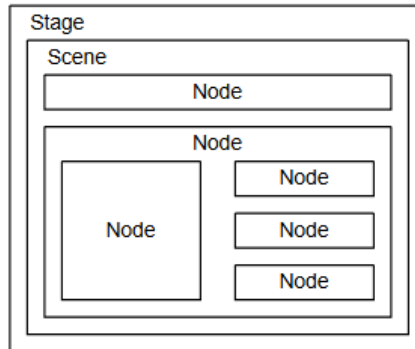
        Stage secondStage = new Stage();      // Create a new Stage object
        secondStage.setTitle("Secomd window");
        secondStage.show();
    }
}

public static void main(String[] args) {
    launch(args); // Launch the GUI
}
```



Basic Structure of JavaFX:

- **Stage:** Top-level container, representing the window itself.
- **Scene:** Physical container for all contents displayed within the Stage.
- **Node:** Visual contents within the scene, such as a button.



Scene (`javafx.scene.Scene`): Mandatory to hold all visual elements.

```
@Override // Override the abstract method in Application class
public void start(Stage primaryStage) {
    Button button = new Button("Click Me!");

    // Create a scene with width=500 and height=400, containing the button node
    Scene scene = new Scene(button, 500, 400);
    primaryStage.setTitle("Main window"); // Set the title
    primaryStage.setScene(scene);         // Place the scene in the stage
    primaryStage.show();                  // Display the stage
}
```

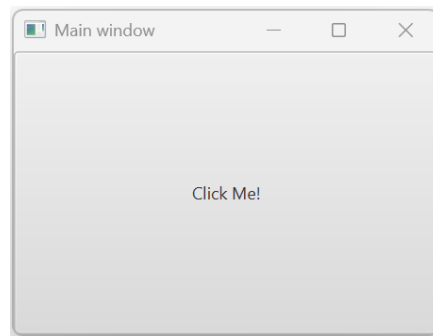


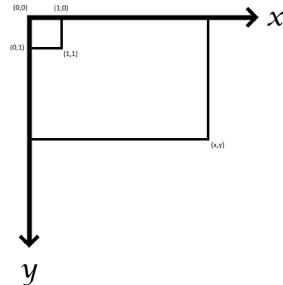
Figure 12: Entire scene is a button

Node: Can be a shape, a UI control, an image view, a group, or a pane.

- **Shape:** A text, line, circle, ellipse, rectangle, arc, polygon, polyline, etc.
- **UI control:** A label, button, check box, radio button, text field, text area, etc.
- **Image view:** A place to display an image
- **Group/Pane:** Container for multiple nodes.

Java **Coordinate System:** Origin (0,0) at top left corner of the window.

- X-axis: x-coordinate increases as node move to the right
- Y-axis: y-coordinate increases as node move down



There are different node types:

- **Parents:** Can be directly placed on a scene.
- **Non-parent nodes:** Have to be placed on a parent node. In most of the time, a group or pane is needed.
- **Branch node:** Can include other nodes as its sub-nodes
- **Leaf node:** Cannot have sub-nodes

	Parent (<code>javafx.scene.Parent</code>)	Non-parent node
Branch node	Group, Pane	-
Leaf node	UI controls	Shape, Image View

Group class (`javafx.scene.Group`): Group a collection of nodes together
UML Diagram of Group (omitted the getter and setter methods for property value):

<code>javafx.scene.Group</code>	
<code>-autoSizeChildren: BooleanProperty</code>	Controls whether the Group resizes any resizable children
<code>+Group()</code>	Creates a group
<code>+Group(children: Node...)</code>	Creates a Group with given children
<code>+getChildren(): ObservableList<Node></code>	Gets the list of children of this Group.

Observable List class (`javafx.collections.ObservableList<E>`): A reference list of items shown in the node.
In the list, E means any non-primitive type.

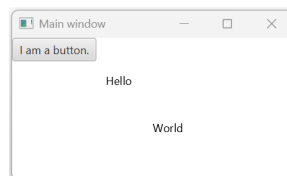
<code>javafx.collections.ObservableList<E></code>	
<code>+sorted(): SortedList<E></code>	Creates a SortedList wrapper with natural sorting
<code>+contains(o Object): boolean</code>	Returns true if the list contains the specified element
<code>+add(e: E): boolean</code>	Appends an element to the end
<code>+add(index: int, element: E): void</code>	Inserts an element at specified position
<code>+addAll(elements: E...): boolean</code>	Append all the elements to the end
<code>+set(index: int, element: E): E</code>	Replaces the element at specified position
<code>+setAll(elements: E...): boolean</code>	Clears the list and add all the elements
<code>+retainAll(elements: E...): boolean</code>	Retains only the elements specified in the list
<code>+remove(from: int, to: int): void</code>	Removes all the elements from start index (inclusive) to end index (exclusive)
<code>+removeAll(elements: E...): boolean</code>	Removes all the elements specified

We can finally show more than one nodes by adding them in the group.

```
@Override
public void start(Stage primaryStage) {
    // Create the nodes
    Button button = new Button("I am a button.");
    Text hello = new Text(100, 50, "Hello");
    Text world = new Text();
    world.setX(150);
    world.setY(100);
    world.setText("World");

    // Create a group to hold more than one node
    Group group = new Group();
    group.getChildren().add(button);           // Add a node to the group
    group.getChildren().addAll(hello, world); // Add multiple nodes to the group at once

    Scene scene = new Scene(group, 300, 150);
    primaryStage.setTitle("Main window");
    primaryStage.setScene(scene);
    primaryStage.show();
}
```



By default, everything is placed at the upper-left corner of the group, overlapping.
This works well when the nodes are placed in specific positions, not for nodes such as buttons.

Text class (`javafx.scene.text.Text`): Shape that displays text

UML Diagram of Text (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.text.Text</code>	
<code>-text: StringProperty</code> <code>-x: DoubleProperty</code> <code>-y: DoubleProperty</code> <code>-underline: BooleanProperty</code> <code>-strikethrough: BooleanProperty</code> <code>-font: ObjectProperty</code>	Defines the displayed text Defines the x-coordinate of text (default: 0) Defines the y-coordinate of text (default: 0) Defines if text is underlined (default: false) Defines if there is a line through the text (default: false) Defines the font of the text
<code>+Text()</code> <code>+Text(text: String)</code> <code>+Text(x: double, y: double, text: String)</code>	Creates a Text

Font class (`javafx.scene.text.Font`): Defines the font for text

Some classes contains the setter method to alter how the text is displayed.

For example, `setFont(Font font)` in the Text class.

UML Diagram of Font (omitted the getter methods for property values):

<code>javafx.scene.text.Font</code>	
<code>-size: double</code> <code>-name: String</code> <code>-family: String</code>	Size of this font Name of this font Family of this font
<code>+Font(size: double)</code> <code>+Font(name: String, size: double)</code> <code>+font(name: String, size: double): Font</code> <code>+font(name: String, w: FontWeight, size: double): Font</code> <code>+font(name: String, w: FontWeight, p: FontPosture, size: double): Font</code> <code>+getFamilies(): List<String></code> <code>+getFontNames(): List<String></code> <code>+getFontNames(family: String): List<String></code>	Creates a Font Creates a Font Returns a list of font family names Returns a list of full font names including family and weight Returns a list of font names for the given family

Circle class (`javafx.scene.shape.Circle`): Shape that draws a circle

UML Diagram of Circle (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.shape.Circle</code>	
<code>-centerX: DoubleProperty</code>	Defines the x-coordinate of the circle's center
<code>-centerY: DoubleProperty</code>	Defines the y-coordinate of the circle's center
<code>-radius: DoubleProperty</code>	Defines the radius of the circle
<code>+Circle()</code> <code>+Circle(radius: double)</code> <code>+Circle(radius: double, fill: Paint)</code> <code>+Circle(centerX: double, centerY: double, radius: double)</code> <code>+Circle(centerX: double, centerY: double, radius: double, fill: Paint)</code>	Creates a Circle

Rectangle class (`javafx.scene.shape.Rectangle`): Shape that draws a rectangle

UML Diagram of Rectangle (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.shape.Rectangle</code>	
<code>-x: DoubleProperty</code>	Defines the x-coordinate of the upper-left corner of the rectangle
<code>-y: DoubleProperty</code>	Defines the y-coordinate of the upper-left corner of the rectangle
<code>-width: DoubleProperty</code>	Defines the width of the circle (default: 0)
<code>-height: DoubleProperty</code>	Defines the height of the circle (default: 0)
<code>-arcWidth: DoubleProperty</code>	Defines the horizontal diameter of the arcs at the corner (default: 0)
<code>-arcHeight: DoubleProperty</code>	Defines the vertical diameter of the arcs at the corner (default: 0)
<code>+Rectangle()</code> <code>+Rectangle(width: double, height: double)</code> <code>+Rectangle(width: double, height: double, fill: Paint)</code> <code>+Rectangle(x: double, y: double, width: double, height: double)</code>	Creates a Rectangle

There are some other shapes such as Ellipse, Polygon, Path, Line, Arc, etc.

Color class (`javafx.scene.paint.Color`): Defines the color (subclass of the Paint class used to fill the shape)

UML Diagram of Color (omitted the getter methods of property values):

<code>javafx.scene.paint.Color</code>	
<code>-red: double</code>	Red value of the Color (between 0.0 and 1.0)
<code>-green: double</code>	Green value of the Color (between 0.0 and 1.0)
<code>-blue: double</code>	Blue value of the Color (between 0.0 and 1.0)
<code>-opacity: double</code>	Opacity of the Color (between 0.0 and 1.0)
<code>+Color(r: double, g: double, b: double, opacity: double)</code>	Creates a Color with specified red, green, and blue values in the range from 0.0 to 1.0
<code>+color(r: double, g: double, b: double): Color</code>	Creates a Color with specified red, green, and blue values in the range from 0.0 to 1.0
<code>+color(r: double, g: double, b: double, opacity: double): Color</code>	
<code>+rgb(r: int, g: int, b: int): Color</code>	Creates a Color with specified red, green, and blue values in the range from 0 to 255
<code>+rgb(r: int, g: int, b: int, opacity: double): Color</code>	
<code>+brighter(): Color</code>	Creates a Color that is a brighter version of this color
<code>+darker(): Color</code>	Creates a Color that is a darker version of this color

ImageView Class (`javafx.scene.image.ImageView`): Displays an image

UML Diagram of ImageView (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.image.ImageView</code>	
<code>-fitHeight: DoubleProperty</code>	Defines the height of the bounding box which the image is resized to fit
<code>-fitWidth: DoubleProperty</code>	Defines the width of the bounding box which the image is resized to fit
<code>-x: DoubleProperty</code>	Defines the x-coordinate of the upper-left corner of the ImageView
<code>-y: DoubleProperty</code>	Defines the y-coordinate of the upper-left corner of the ImageView
<code>-image: ObjectProperty</code>	Defines the image to be painted
<code>+ImageView()</code>	Creates an ImageView
<code>+ImageView(image: Image)</code>	
<code>+ImageView(filenameOrURL: String)</code>	Creates an ImageView with image loaded from specified URL

Image Class (`javafx.scene.image.Image`): Loads an image on the window

UML Diagram of Image (omitted the getter methods for property values and getter for property itself):

<code>javafx.scene.image.Image</code>	
<code>-height: ReadOnlyDoubleProperty</code>	Image height or 0 if the image loading fails
<code>-width: ReadOnlyDoubleProperty</code>	Image width or 0 if the image loading fails
<code>-progress: ReadOnlyDoubleProperty</code>	Approximate percentage of image's loading completed
<code>-error: ReadOnlyBooleanProperty</code>	Indicates whether an error was detected while loading an image
<code>+Image(filenameOrURL: String)</code>	Constructs an Image with content loaded from specified URL

In addition to Group class, we can also use layout panes to group the nodes together in a specific layout.

Pane Class (`javafx.scene.layout.Pane`): Base class for layout panes.

There are other types of panes:

Class	Description
<code>Pane</code>	Base class for layout panes
<code>StackPane</code>	Places the nodes on top of each other in the center of the pane
<code>FlowPane</code>	Places the nodes row-by-row horizontally or column-by-column vertically
<code>GridPane</code>	Places the nodes in the cells in a two-dimensional grid
<code>BorderPane</code>	Places the nodes in the top, right, bottom, left, and center regions
<code>HBox</code>	Places the nodes in a single row
<code>VBox</code>	Places the nodes in a single column

UML Diagram of Pane (only including important items):

<code>javafx.scene.layout.Pane</code>	
<code>+Pane()</code>	Creates a Pane
<code>+Pane(children: Node...)</code>	
<code>+getChildren(): ObservableList<Node></code>	Gets the list of children of this Pane.
<code>+setPadding(value: Insets): void</code>	Sets the padding

Similar to Group, we can add nodes by using the `getChildren()` method.

```
// Add a node to a pane
pane.getChildren().add(node);

// Set the x-coordinate of the circle to the center of the pane
// If the width of the pane changes, the x-coordinate of the circle will also change
circle.centerXProperty().bind(pane.widthProperty().divide(2))
```

Insets Class (`javafx.geometry.Insets`): A set of inside offsets for the four side of a rectangular area. Useful for defining paddings (distance between node and border)

UML Diagram of Insets (omitted getter methods):

<code>javafx.geometry.Insets</code>	
<code>+Insets(topRightBottomLeft: double)</code>	Creates a Insets with same value for all four offsets
<code>+Insets(top: double, right: double, bottom: double, left: double)</code>	Creates a Insets with four different offsets

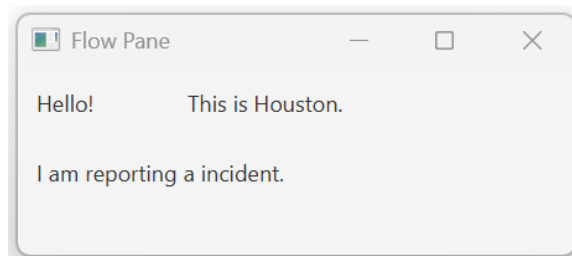
FlowPane (`javafx.scene.layout.FlowPane`): Places the nodes row-by-row horizontally or column-by-column vertically

UML Diagram of FlowPane (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.layout.FlowPane</code>	
<code>-alignment: ObjectProperty<Pos></code>	Overall alignment of the content (default: <code>Pos.LEFT</code>)
<code>-orientation: ObjectProperty<Orientation></code>	Orientation in this pane (default: <code>Orientation.HORIZONTAL</code>)
<code>-hgap: DoubleProperty</code>	Horizontal gap between the sub-nodes (default: 0)
<code>-vgap: DoubleProperty</code>	Vertical gap between the sub-nodes (default: 0)
<code>+FlowPane()</code> <code>+FlowPane(children: Node...)</code> <code>+FlowPane(hgap: double, vgap: double)</code> <code>+FlowPane(hgap: double, vgap: double, children: Node...)</code> <code>+FlowPane(orientation: ObjectProperty<Orientation>)</code> <code>+FlowPane(orientation: ObjectProperty<Orientation>, hgap: double, vgap: double)</code>	Creates a FlowPane

```
@Override
public void start(Stage primaryStage) {
    // Create a flow pane
    FlowPane flowPane = new FlowPane(50, 20); // Hgap: 50, Vgap: 20
    flowPane.setPadding(new Insets(10));      // Padding: 10
    flowPane.getChildren().addAll(
        new Label("Hello!"),
        new Label("This is Houston."),
        new Label("I am reporting an incident.")
    );

    Scene scene = new Scene(flowPane, 300, 100);
    primaryStage.setTitle("Flow Pane");
    primaryStage.setScene(scene);
    primaryStage.show();
}
```



GridPane (javafx.scene.layout.GridPane): Places the nodes in the cells in a two-dimensional grid
UML Diagram of GridPane (omitted the getter and setter methods for property values and getter for property itself):

javafx.scene.layout.GridPane	
-alignment: ObjectProperty<Pos>	Overall alignment of the content (default: Pos.LEFT)
-gridLineVisible: BooleanProperty	Indicates grid line visible or not
-hgap: DoubleProperty	Horizontal gap between the sub-nodes (default: 0)
-vgap: DoubleProperty	Vetical gap between the sub-nodes (default: 0)
+GridPane()	Creates a GridPane
+add(child: Node, columnIndex: int, rowIndex: int): void	Add a node in specified column and row
+addColumn(columnIndex: int, children: Node...): void	Add multiple nodes to specific column
+addRow(rowIndex: int, children: Node...): void	Add multiple nodes to specific row
+getColumnIndex(child: Node): int	Gets the column index for specific node
+setColumnIndex(child: Node, columnIndex: int): void	Sets a node to a new column
+getRowIndex(child: Node): int	Gets the row index for specific node
+setRowIndex(child: Node, rowIndex: int): void	Sets a node to a new row
+setHalignment(child: Node, value: HPos): void	Sets the horizontal alignment for the node
+setValignment(child: Node, value: VPos): void	Sets the vertical alignment for the node

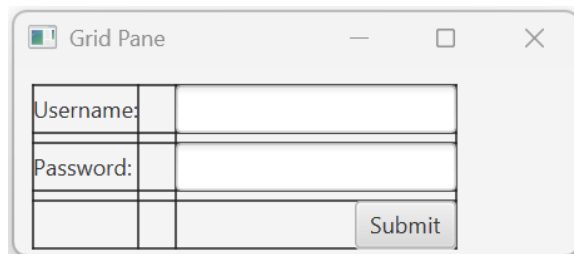
```

@Override
public void start(Stage primaryStage) {
    // Create a grid pane
    GridPane gridPane = new GridPane(20, 5); // Hgap: 20, Vgap: 5
    gridPane.setGridLinesVisible(true);
    gridPane.setPadding(new Insets(10)); // Padding: 10
    gridPane.add(new Label("Username:"), 0, 0);
    gridPane.add(new TextField(), 1, 0);
    gridPane.addRow(1, new Label("Password:"), new TextField());

    Button submitButton = new Button("Submit");
    gridPane.add(submitButton, 1, 2);
    GridPane.setHalignment(submitButton, HPos.RIGHT); // This is a static method

    Scene scene = new Scene(gridPane, 300, 100);
    primaryStage.setTitle("Grid Pane");
    primaryStage.setScene(scene);
    primaryStage.show();
}

```



BorderPane (`javafx.scene.layout.BorderPane`): Places the nodes in the top, right, bottom, left, and center regions
UML Diagram of BorderPane (omitted the getter and setter methods for property values and getter for property itself):

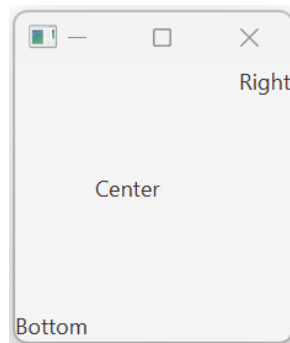
javafx.scene.layout.BorderPane	
-top: ObjectProperty<Node>	Node placed in the top region (default: null)
-right: ObjectProperty<Node>	Node placed in the right region (default: null)
-bottom: ObjectProperty<Node>	Node placed in the bottom region (default: null)
-left: ObjectProperty<Node>	Node placed in the left region (default: null)
-center: ObjectProperty<Node>	Node placed in the center region (default: null)
+BorderPane()	Creates a BorderPane
+BorderPane(center: Node)	
+BorderPane(center: Node, top: Node, right: Node, bottom: Node, left: Node)	
+setAlignment(child: Node, pos: Pos): void	Sets the alignment for the node (static method)

```

@Override
public void start(Stage primaryStage) {
    // Create a border pane
    BorderPane borderPane = new BorderPane(20, 5); // Hgap: 20, Vgap: 5
    borderPane.setRight(new Label("Right"));
    borderPane.setBottom(new Label("Bottom"));
    borderPane.setCenter(new Label("Center"));

    Scene scene = new Scene(borderPane, 150, 105);
    primaryStage.setTitle("Border Pane");
    primaryStage.setScene(scene);
    primaryStage.show();
}

```



HBox (`javafx.scene.layout.HBox`): Lays out the children in a single horizontal row.

UML Diagram of HBox (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.layout.HBox</code>	
<code>-alignment: ObjectProperty<Pos></code> <code>-fillHeight: BooleanProperty</code>	Overall alignment of the children (default: <code>Pos.TOP_LEFT</code>) Indicates whether resizable children fill the full height of the box (default: <code>true</code>)
<code>-spacing: DoubleProperty</code>	Horizontal gap between two nodes (default: 0)
<code>+HBox()</code> <code>+HBox(children: Node...)</code> <code>+HBox(spacing: double)</code> <code>+HBox(spacing: double, children: Node...)</code> <code>+setMargin(node: Node, value: Insets): void</code>	Creates a HBox Sets the margin for the sub-nodes

VBox (`javafx.scene.layout.VBox`): Lays out the children in a single vertical column.

UML Diagram of HBox (omitted the getter and setter methods for property values and getter for property itself):

<code>javafx.scene.layout.VBox</code>	
<code>-alignment: ObjectProperty<Pos></code> <code>-fillWidth: BooleanProperty</code>	Overall alignment of the children (default: <code>Pos.TOP_LEFT</code>) Indicates whether resizable children fill the full width of the box (default: <code>true</code>)
<code>-spacing: DoubleProperty</code>	Vertical gap between two nodes (default: 0)
<code>+VBox()</code> <code>+VBox(children: Node...)</code> <code>+VBox(spacing: double)</code> <code>+VBox(spacing: double, children: Node...)</code> <code>+setMargin(node: Node, value: Insets): void</code>	Creates a VBox Sets the margin for the sub-nodes

13 Abstract Class

Abstract method: Does not have the method body. Declared with the **abstract** modifier.

Syntax: `abstract returnType methodName(parameterList)`

- If all subclasses need the method with same method body, we can declare it a regular method in the superclass. All subclasses inherit it automatically.
- If all subclasses need the method with different method body, we can declare it an abstract method in the superclass. All subclasses override it manually.

Abstract class: The class contains some abstract methods. Declared with the **abstract** modifier.

The class must be declared as abstract when it includes an abstract method.

- It is a restricted class that cannot be used to create objects (`new AbstractClass()` is not allowed)
- It can contain constructors, which is called by the subclasses.
- It does not change the inheritance rule. It can be used as the type of a reference variable.
- Abstract methods cannot be static or private because a static or private method cannot be overridden.
- If a subclass inherits abstract method from superclass but doesn't override it, the subclass needs to be abstract as well.

Interface: Declared with the **interface** keyword.

Interface syntax: `public interface interfaceName {...}`

Implementing an interface syntax:

`public class className implements interfaceName {...}`

`public class subClassName extends superClassName implements interfaceName {...}`

- If an interface has abstract methods, the class implementing it must override all abstract methods inherited.
- A subclass can only have one superclass, but it can implements multiple interfaces, separated by comma.
- Interface acts as a superclass. If a superclass and an interface have the same method, the one in the superclass is chosen.
- Interface cannot hold instance variables and constructors.
- Interface can extends other interfaces, but it cannot implements any interfaces.
- Interfaces with same class variables can be implemented together, which we access by `interfaceName.dataFieldName`.
- Interfaces with same static method can be implemented together, which we access by `interfaceName.methodName(...)`.
- Interfaces having non-static methods with the same signature and compatible return value types can be implemented together, but we need to override it. It is accessed by `interfaceName.super.methodName(...)`.
If the return types are incompatible, the interfaces cannot be implemented together.

Comparable Class: Allows comparisons between two objects. When implementing it, we need to add a class after it.

Syntax: `className implements Comparable <className> {...}`

- You needs to override the abstract `compareTo()` method inside the Comparable class:
`public int compareTo(className object2) {...}`
- In numerical sense:
 - `object1 < object2`: Returns negative
 - `object1 = object2`: Returns zero
 - `object1 > object2`: Returns positive
- This is useful when we use `Arrays.sort()` for sorting arrays. We can claim that the class is comparable.

14 Event-driven Programming

Procedural Programming: Code is executed in procedural order

Event-driven Programming: Code is executed upon activation of events

Event source object: The object where the event is triggered.

GUI elements usually have a set of event-handler properties named in a way like "on"+*eventName*.

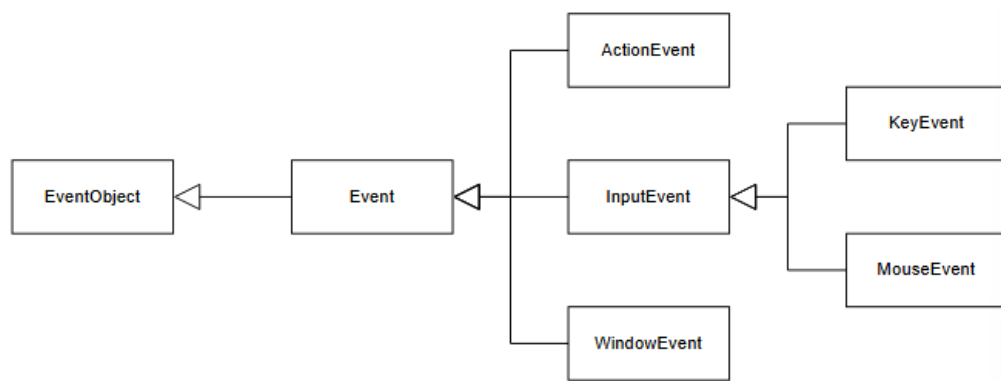
The type of event-handler property is an event-handler class, which defines what to do when a certain event is fired.

JavaFX has a unified handler interface `EventHandler<EventType>` containing an abstract method `handle(EventType e)`.

When an event happens, Java automatically create an event object to store the information about the event. GUI events usually have the similar data flow:

1. Clicking a button (event source object) fires an action event.
2. An event is an object.
3. The event handler processes the event.

There are different event classes describing different events, which the most notable one is shown below:



We can get the event source object which the Event initially occurred by the `getSource()` method.

Common source objects and event types:

User Action	Source Object	Event Type Fired	Event Registration Method
Click a button	Button	ActionEvent	setOnAction(EventHandler<ActionEvent>)
Press Enter in a text field	TextField		
Check or uncheck	RadioButton		
Check or uncheck	CheckBox		
Select a new item	ComboBox		
Mouse pressed	Node, Scene	MouseEvent	setOnMousePressed(EventHandler<MouseEvent>)
Mouse released			setOnMouseReleased(EventHandler<MouseEvent>)
Mouse clicked			setOnMouseClicked(EventHandler<MouseEvent>)
Mouse entered			setOnMouseEntered(EventHandler<MouseEvent>)
Mouse exited			setOnMouseExited(EventHandler<MouseEvent>)
Mouse moved			setOnMouseMoved(EventHandler<MouseEvent>)
Mouse dragged			setOnMouseDragged(EventHandler<MouseEvent>)
Key pressed			setOnKeyPressed(EventHandler<KeyEvent>)
Key released	Node, Scene	KeyEvent	setOnKeyReleased(EventHandler<KeyEvent>)
Key typed			setOnKeyTyped(EventHandler<KeyEvent>)

Event handler class is designed for a specific node in a specific class to handle event in tailor-made and customized manner. We define the event handler class as an **inner class** as a member of the **outer class** like data fields and methods.

- Inner class can directly access the data fields and methods defined in the outer class.
- Inner class can be defined as static. Static inner class can be used in other classes.
Syntax: *OuterClassName.InnerClassName varName = OuterClassName.new InnerClassName()*
- Inner class can use visibility modifiers. Non-private inner class can be used in other classes (subjected to visibility).
Syntax: *OuterClassName.InnerClassName varName = OuterClassObjectName.new InnerClassName()*
- Inner class is compiled to *OuterClassName\$InnerClassName.class*

```
public class OuterClass {
    private int num;
    public void func() {...}
    private class InnerClass { // Allow private because it is a member of outer class
        public void infunc() {
            num++;
            func();
        }
    }
}
```

An event-handler class is often written for one node, and thus creating a class formally is not necessary.

Anonymous Inner Classes: Inner class without a name. Combining declaring an inner class and creating an instance.

Syntax:

```
new SuperClassName/InterfaceName() {
    // Implement or override methods
}
```

- This class is only used at the declaration location. It will create an object at the same time.
- The anonymous inner class still needs to override all abstract methods in the superclass or the interface.

Lambda Expression: Anonymous method

Syntax:

```
(parameterList) -> {
    statements
}
```

- In the parameter list, data type for a parameter may be explicitly declared or implicitly inferred by the compiler.
- Parentheses can be omitted if there is only one parameter without an explicit data type.

Single Abstract Method (SAM) interface / functional interface: Interface with exactly one abstract method

Function object: Object that performs a function by invoking single method.

If the interface is a SAM interface, instead of using anonymous inner class, we can directly use lambda expression.

If the inner class have multiple abstract methods, we cannot use lambda expression to simplify.

```
btn.setOnAction(
    new EventHandler<ActionEvent>() {
        @Override
        public void handle(ActionEvent e) {
            // Processing event e
        }
    }
)

btn.setOnAction(e -> {
    // Processing event e
})
```

Figure 13: EventHandler interface contains exactly one abstract method

MouseEvent Class (`javafx.scene.input.MouseEvent`): Handle mouse events

UML Diagram of MouseEvent:

<code>javafx.scene.input.MouseEvent</code>	
<code>+getButton(): MouseButton</code>	Indicates which mouse button has been clicked
<code>+getClickCount(): int</code>	Returns the number of mouse clicks
<code>+getX(): double</code>	Returns the x-coordinate of the mouse point in the event source node
<code>+getY(): double</code>	Returns the y-coordinate of the mouse point in the event source node
<code>+getSceneX(): double</code>	Returns the x-coordinate of the mouse point in the scene
<code>+getSceneY(): double</code>	Returns the y-coordinate of the mouse point in the scene
<code>+getScreenX(): double</code>	Returns the x-coordinate of the mouse point in the screen
<code>+getScreenY(): double</code>	Returns the y-coordinate of the mouse point in the screen
<code>+isAltDown(): boolean</code>	Returns true if the Alt key is pressed
<code>+isControlDown(): boolean</code>	Returns true if the Control key is pressed
<code>+isMetaDown(): boolean</code>	Returns true if the mouse Meta button is pressed
<code>+isShiftDown(): boolean</code>	Returns true if the Shift key is pressed

KeyEvent Class (`javafx.scene.input.KeyEvent`): Enables the use of keys to control and perform actions, or get input from keyboard.

- When the EventHandler is registered to a node, we need to let the node get a focus by using `nodeName.requestFocus()`.

```
textField.setFocusTraversable(true); // Make sure the node can accept focus
textField.setOnKeyPressed(e -> {
    if (event.getCode() == KeyCode.ENTER) {
        ...
    }
})
stage.show();
textField.requestFocus();
```

UML Diagram of KeyEvent:

<code>javafx.scene.input.KeyEvent</code>	
<code>+getCharacter(): String</code>	Returns the character associated with the key
<code>+getCode(): KeyCode</code>	Returns the key code associated with the key
<code>+getText(): String</code>	Returns a string describing the key code
<code>+isAltDown(): boolean</code>	Returns true if the Alt key is pressed
<code>+isControlDown(): boolean</code>	Returns true if the Control key is pressed
<code>+isMetaDown(): boolean</code>	Returns true if the mouse Meta button is pressed
<code>+isShiftDown(): boolean</code>	Returns true if the Shift key is pressed

KeyCode Constants: An enum-type object. It represents a group of constants.

Constant	Description	Constant	Description
HOME	Home key	CONTROL	Control Key
END	End key	SHIFT	Shift key
PAGE_UP	Page Up key	BACK_SPACE	Backspace key
PAGE_DOWN	Page Down key	CAPS	Caps Lock key
UP	Up-arrow key	NUM_LOCK	Num Lock key
DOWN	Down-arrow key	ENTER	Enter Key
LEFT	Left-arrow key	UNDEFINED	<code>keyCode</code> unknown
RIGHT	Right-arrow key	F1 to F12	Function keys from F1 to F12
ESCAPE	Esc key	0 to 9	Number keys from 0 to 9
TAB	Tab key	A to Z	Letter keys from A to Z

Listener Class: Handles when the property of a node changes.

- It implements the `InvalidationListener` interface, which has one abstract method `invalidate(Observable ov)`.
- Every binding property has the `addListener(InvalidationListener listener)` method.
- Objects that add a listener must implement the `Observable` interface. Every binding property is an `Observable` object.